

L04 Computing NE in two player games

CS 280 Algorithmic Game Theory

Ioannis Panageas

For known support

Question: Suppose we knew the **supports** of a Nash equilibrium. Can we compute one?

For known support

Question: Suppose we knew the **supports** of a Nash equilibrium. Can we compute one?

Answer: Yes, via **Linear Programming!**

Let R, C be the payoff matrices of the row and column players, of size $n \times m$.

For known support

Question: Suppose we knew the **supports** of a Nash equilibrium. Can we compute one?

Answer: Yes, via **Linear Programming!**

Let R, C be the payoff matrices of the row and column players, of size $n \times m$.

Any Nash equilibrium (x, y) with **supports** S, T must **satisfy**:

1a) $x_i \geq 0$ for all $i \in [n]$.

2a) $x_i = 0$ for all $i \notin S$.

3a) $\sum_{i \in S} x_i = 1$.

1b) $y_i \geq 0$ for all $i \in [m]$.

2b) $y_i = 0$ for all $i \notin T$.

3b) $\sum_{i \in T} y_i = 1$.

For known support

Question: Suppose we knew the **supports** of a Nash equilibrium. Can we compute one?

Answer: Yes, via **Linear Programming!**

Let R, C be the payoff matrices of the row and column players, of size $n \times m$.

Any Nash equilibrium (x, y) with **supports** S, T must **satisfy**:

1a) $x_i \geq 0$ for all $i \in [n]$.

1b) $y_i \geq 0$ for all $i \in [m]$.

2a) $x_i = 0$ for all $i \notin S$.

2b) $y_i = 0$ for all $i \notin T$.

3a) $\sum_{i \in S} x_i = 1$.

3b) $\sum_{i \in T} y_i = 1$.

4a) $(Ry)_i \geq (Ry)_j \quad \forall i \in S, j \in [n]$.

4b) $(C^\top x)_i \geq (C^\top x)_j \quad \forall i \in T, j \in [m]$.

Any feasible solution is a Nash equilibrium.

Its supports are contained in S and T .

A trivial algorithm

LP (S, T)

$$(C^\top x)_i \geq (C^\top x)_j \quad \forall i \in T, j \in [m].$$

$$(Ry)_i \geq (Ry)_j \quad \forall i \in S, j \in [n].$$

$$\sum_{i \in S} x_i = 1.$$

$$\sum_{i \in T} y_i = 1.$$

$$x_i = 0 \text{ for all } i \notin S.$$

$$y_i = 0 \text{ for all } i \notin T.$$

$$x_i \geq 0 \text{ for all } i \in [n].$$

$$y_i \geq 0 \text{ for all } i \in [m].$$

Algorithm: For all index sets S, T , check feasibility of $LP(S, T)$. If a feasible solution (x, y) is found, it is a Nash equilibrium.

Any feasible solution is a Nash equilibrium.

Its supports are contained in S and T .

A trivial algorithm

LP (S, T)

$$(C^\top x)_i \geq (C^\top x)_j \quad \forall i \in T, j \in [m].$$

$$(Ry)_i \geq (Ry)_j \quad \forall i \in S, j \in [n].$$

$$\sum_{i \in S} x_i = 1.$$

$$\sum_{i \in T} y_i = 1.$$

$$x_i = 0 \text{ for all } i \notin S.$$

$$y_i = 0 \text{ for all } i \notin T.$$

$$x_i \geq 0 \text{ for all } i \in [n].$$

$$y_i \geq 0 \text{ for all } i \in [m].$$

Running time $2^{n+m} \cdot \text{poly}(n, m, L)!$

L : bit length of the rational payoff input.

Slow, not polynomial!

Algorithm: For all index sets S, T , check feasibility of LP (S, T) . If a feasible solution (x, y) is found, it is a Nash equilibrium.

Any feasible solution is a Nash equilibrium.

Its supports are contained in S and T .

Lemke-Howson Algorithm

Assumption: Matrices R, C have strictly positive entries. No loss of generality,

NE are invariant under shifting.

Basic idea: The Lemke-Howson algorithm maintains a single **guess of the supports**, and in **each iteration** we change the guess only a little bit.

Lemke-Howson Algorithm

Assumption: Matrices R, C have strictly positive entries. No loss of generality,

NE are invariant under shifting.

Basic idea: The Lemke-Howson algorithm maintains a single **guess of the supports**, and in **each iteration** we change the guess only a little bit.

$$P_1 = \{x \in \mathbb{R}^n : \forall i \in [n] \ x_i \geq 0 \ \& \ \forall j \in [m] \ (x^\top C)_j \leq 1\}.$$

$$P_2 = \{y \in \mathbb{R}^m : \forall i \in [m] \ y_i \geq 0 \ \& \ \forall j \in [n] \ (Ry)_j \leq 1\}.$$

$$\text{nrml}(x) = \left(\sum_{i \in [n]} x_i\right)^{-1} x, \quad x \neq \mathbf{0} \quad \text{nrml}(y) = \left(\sum_{j \in [m]} y_j\right)^{-1} y, \quad y \neq \mathbf{0}$$

Lemke-Howson Algorithm

Assumption: Matrices R, C have strictly positive entries. No loss of generality,

NE are invariant under shifting.

Basic idea: The Lemke-Howson algorithm maintains a single **guess of the supports**, and in **each iteration** we change the guess only a little bit.

$$P_1 = \{x \in \mathbb{R}^n : \forall i \in [n] \ x_i \geq 0 \ \& \ \forall j \in [m] \ (x^\top C)_j \leq 1\}.$$

$$P_2 = \{y \in \mathbb{R}^m : \forall i \in [m] \ y_i \geq 0 \ \& \ \forall j \in [n] \ (Ry)_j \leq 1\}.$$

$$\text{nrml}(x) = \left(\sum_{i \in [n]} x_i\right)^{-1} x, \quad x \neq \mathbf{0} \quad \text{nrml}(y) = \left(\sum_{j \in [m]} y_j\right)^{-1} y, \quad y \neq \mathbf{0}$$

Labels. For $i \in [n]$ and $j \in [m]$:

x : label i if $x_i = 0$; label $n + j$ if $(x^\top C)_j = 1$.

y : label i if $(Ry)_i = 1$; label $n + j$ if $y_j = 0$.

Lemke-Howson Algorithm

Assumption: Matrices R, C have strictly positive entries. No loss of generality,

NE are invariant under shifting.

Basic idea: The Lemke-Howson algorithm maintains a single **guess of the supports**, and in **each iteration** we change the guess only a little bit.

$$P_1 = \{x \in \mathbb{R}^n : \forall i \in [n] \ x_i \geq 0 \ \& \ \forall j \in [m] \ (x^\top C)_j \leq 1\}.$$

$$P_2 = \{y \in \mathbb{R}^m : \forall i \in [m] \ y_i \geq 0 \ \& \ \forall j \in [n] \ (Ry)_j \leq 1\}.$$

$$\text{nrml}(x) = \left(\sum_{i \in [n]} x_i\right)^{-1} x, \quad x \neq \mathbf{0} \quad \text{nrml}(y) = \left(\sum_{j \in [m]} y_j\right)^{-1} y, \quad y \neq \mathbf{0}$$

Labels. For $i \in [n]$ and $j \in [m]$:

x : label i if $x_i = 0$; label $n + j$ if $(x^\top C)_j = 1$.

y : label i if $(Ry)_i = 1$; label $n + j$ if $y_j = 0$.

Lemma. Let $x^* \in P_1$ and $y^* \in P_2$ be nonzero vectors whose labels together cover $1, \dots, n + m$. Then $(\text{nrml}(x^*), \text{nrml}(y^*))$ is a Nash equilibrium.

Lemke-Howson Algorithm

Lemma. *Let $x^* \in P_1$ and $y^* \in P_2$ be nonzero vectors whose labels together cover $1, \dots, n + m$. Then $(\text{nrml}(x^*), \text{nrml}(y^*))$ is a Nash equilibrium.*

Proof.

- For each $i \in [n]$, either $x_i^* = 0$ or $(Ry^*)_i = 1$ (i is best response of row player to $\text{nrml}(y^*)$).
- For each $j \in [m]$, either $y_j^* = 0$ or $(x^{*\top}C)_j = 1$ (j is best response of column player to $\text{nrml}(x^*)$).

Lemke-Howson Algorithm

Lemma. Let $x^* \in P_1$ and $y^* \in P_2$ be nonzero vectors whose labels together cover $1, \dots, n + m$. Then $(\text{nrml}(x^*), \text{nrml}(y^*))$ is a Nash equilibrium.

Proof.

- For each $i \in [n]$, either $x_i^* = 0$ or $(Ry^*)_i = 1$ (i is best response of row player to $\text{nrml}(y^*)$).
- For each $j \in [m]$, either $y_j^* = 0$ or $(x^{*\top} C)_j = 1$ (j is best response of column player to $\text{nrml}(x^*)$).

We conclude that

$$\text{if } x_i^* > 0 \Rightarrow (Ry^*)_i \geq (Ry^*)_j \quad \forall j \in [n]$$

$$\text{if } y_j^* > 0 \Rightarrow (x^{*\top} C)_j \geq (x^{*\top} C)_i \quad \forall i \in [n]$$

Lemke-Howson Algorithm

Lemma. Let $x^* \in P_1$ and $y^* \in P_2$ be nonzero vectors whose labels together cover $1, \dots, n + m$. Then $(\text{nrml}(x^*), \text{nrml}(y^*))$ is a Nash equilibrium.

Proof.

- For each $i \in [n]$, either $x_i^* = 0$ or $(Ry^*)_i = 1$ (i is best response of row player to $\text{nrml}(y^*)$).
- For each $j \in [m]$, either $y_j^* = 0$ or $(x^{*\top} C)_j = 1$ (j is best response of column player to $\text{nrml}(x^*)$).

We conclude that

$$\text{if } x_i^* > 0 \Rightarrow (Ry^*)_i \geq (Ry^*)_j \quad \forall j \in [n]$$

$$\text{if } y_j^* > 0 \Rightarrow (x^{*\top} C)_j \geq (x^{*\top} C)_i \quad \forall i \in [n]$$

Hence the same is true for $\text{nrml}(x^*), \text{nrml}(y^*)$.

Lemke-Howson Algorithm

Lemma. Let $x^* \in P_1$ and $y^* \in P_2$ be nonzero vectors whose labels together cover $1, \dots, n + m$. Then $(\text{nrml}(x^*), \text{nrml}(y^*))$ is a Nash equilibrium.

Proof.

- For each $i \in [n]$, either $x_i^* = 0$ or $(Ry^*)_i = 1$ (i is best response of row player to $\text{nrml}(y^*)$).
- For each $j \in [m]$, either $y_j^* = 0$ or $(x^{*\top} C)_j = 1$ (j is best response of column player to $\text{nrml}(x^*)$).

We conclude that

$$\text{if } x_i^* > 0 \Rightarrow (Ry^*)_i \geq (Ry^*)_j \quad \forall j \in [n]$$

$$\text{if } y_j^* > 0 \Rightarrow (x^{*\top} C)_j \geq (x^{*\top} C)_i \quad \forall i \in [m]$$

Hence the same is true for $\text{nrml}(x^*), \text{nrml}(y^*)$.

The normalized pair satisfies $\text{LP}(\text{Supp}(x^*), \text{Supp}(y^*))$!

Conversely, every Nash equilibrium gives a fully labeled pair after rescaling by the players' positive equilibrium payoffs.

Lemke-Howson Algorithm

Definition (**Vertices and nondegeneracy**). Assume the game is nondegenerate. Every vertex of P_1 has exactly n tight, linearly independent constraints, and every vertex of P_2 has exactly m . All remaining inequalities are strict. A vertex of $P_1 \times P_2$ therefore has $n + m$ tight constraints.

Lemke-Howson Algorithm

Definition (Vertices and nondegeneracy). Assume the game is nondegenerate. Every vertex of P_1 has exactly n tight, linearly independent constraints, and every vertex of P_2 has exactly m . All remaining inequalities are strict. A vertex of $P_1 \times P_2$ therefore has $n + m$ tight constraints.

Algorithm (Lemke-Howson). We define the following algorithm:

1. Initialize $x = \mathbf{0}$ and $y = \mathbf{0}$.
2. Set $k = k_0 = 1$.
3. **Loop**
4. In P_1 , drop label k and pivot to the neighboring vertex x' . Let k' be the new label.
5. **Set** $x = x'$.
6. **If** $k' = k_0$, **return** $(\text{nrml}(x), \text{nrml}(y))$.
7. In P_2 , drop label k' and pivot to the neighboring vertex y' . Let k'' be the new label.
8. **Set** $y = y'$.
9. **If** $k'' = k_0$, **return** $(\text{nrml}(x), \text{nrml}(y))$.
10. **Set** $k = k''$ and repeat.

Analysis of Lemke-Howson

Theorem. *The Lemke-Howson algorithm outputs a Nash equilibrium.*

Proof. Fix dropped label 1. Graph vertices are vertex pairs $(x, y) \in P_1 \times P_2$ whose combined labels contain $2, \dots, n + m$. Edges pivot in one polytope while preserving these labels.

- **One duplicate label**, with label 1 missing. There are exactly two neighbors: drop the duplicate from x and pivot in P_1 , or drop it from y and pivot in P_2 .

Analysis of Lemke-Howson

Theorem. *The Lemke-Howson algorithm outputs a Nash equilibrium.*

Proof. Fix dropped label 1. Graph vertices are vertex pairs $(x, y) \in P_1 \times P_2$ whose combined labels contain $2, \dots, n + m$. Edges pivot in one polytope while preserving these labels.

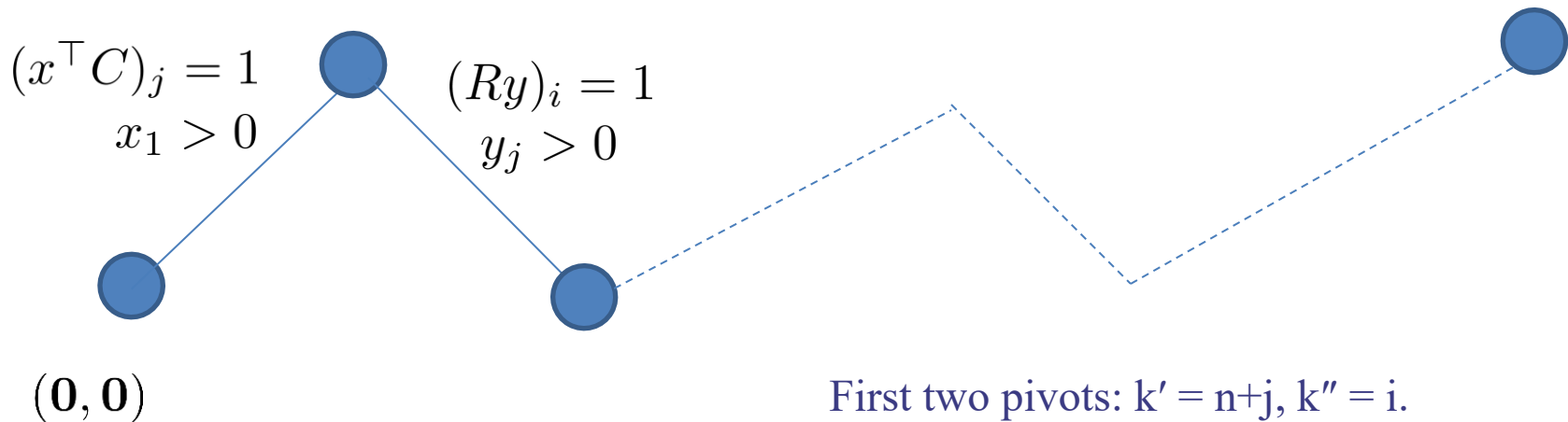
- **One duplicate label**, with label 1 missing. There are exactly two neighbors: drop the duplicate from x and pivot in P_1 , or drop it from y and pivot in P_2 .
- **All labels** occur exactly once. There is exactly one neighbor: drop label 1 from whichever vector has it.

Analysis of Lemke-Howson

Theorem. *The Lemke-Howson algorithm outputs a Nash equilibrium.*

Proof. Fix dropped label 1. Graph vertices are vertex pairs $(x, y) \in P_1 \times P_2$ whose combined labels contain $2, \dots, n+m$. Edges pivot in one polytope while preserving these labels.

- **One duplicate label**, with label 1 missing. There are exactly two neighbors: drop the duplicate from x and pivot in P_1 , or drop it from y and pivot in P_2 .
- **All labels** occur exactly once. There is exactly one neighbor: drop label 1 from whichever vector has it.



Analysis of Lemke-Howson

Proof cont. Since each vertex in this finite graph has degree 1 or 2, the graph is a **union of cycles and paths!**

Analysis of Lemke-Howson

Proof cont. Since each vertex in this finite graph has degree 1 or 2, the graph is a **union of cycles and paths!**

1. Lemke-Howson algorithm **begins** at the configuration **$(0, 0)$** .
2. $(0, 0)$ has **all labels** and is therefore an endpoint of a path component.
3. The algorithm will terminate at the other endpoint of the path.
4. The other point is not $(0, 0)$ and cannot be $(x, 0)$ or $(0, y)$.

Analysis of Lemke-Howson

Proof cont. Since each vertex in this finite graph has degree 1 or 2, the graph is a **union of cycles and paths!**

1. Lemke-Howson algorithm **begins** at the configuration **$(0, 0)$** .
2. $(0, 0)$ has **all labels** and is therefore an endpoint of a path component.
3. The algorithm will terminate at the other endpoint of the path.
4. The other point is not $(0, 0)$ and cannot be $(x, 0)$ or $(0, y)$.

Normalize the two endpoint vectors.
The previous lemma gives a Nash equilibrium!

Other facts

Corollary (Odd Number). *For non-degenerate games, the number of Nash equilibria is **odd**!*

Other facts

Corollary (Odd Number). *For non-degenerate games, the number of Nash equilibria is **odd**!*

Proof. In a graph, the number of vertices of odd degree is even since

$$\sum_v d_v = 2E.$$

Other facts

Corollary (Odd Number). *For non-degenerate games, the number of Nash equilibria is **odd**!*

Proof. In a graph, the number of vertices of odd degree is even since

$$\sum_v d_v = 2E.$$

Hence an even number of vertices have odd degree. These are exactly $(\mathbf{0}, \mathbf{0})$ and the pairs corresponding to Nash equilibria. Removing $(\mathbf{0}, \mathbf{0})$ leaves an odd number of equilibria.

Other facts

Corollary (Odd Number). *For non-degenerate games, the number of Nash equilibria is **odd**!*

Proof. In a graph, the number of vertices of odd degree is even since

$$\sum_v d_v = 2E.$$

Hence an even number of vertices have odd degree. These are exactly $(\mathbf{0}, \mathbf{0})$ and the pairs corresponding to Nash equilibria. Removing $(\mathbf{0}, \mathbf{0})$ leaves an odd number of equilibria.

Theorem (Savani, von Stengel'04). *The Lemke–Howson algorithm runs in exponential time in the worst case.*

Approximating a Nash eq.

Definition (*k -uniform*). Let $k \geq 1$ be an integer. A strategy x is k -uniform if every coordinate x_i is a multiple of $1/k$.

Observation: A k -uniform strategy has support size at most k .

Approximating a Nash eq.

Definition (***k-uniform***). Let $k \geq 1$ be an integer. A strategy x is k -uniform if every coordinate x_i is a multiple of $1/k$.

Observation: A k -uniform strategy has support size at most k .

Theorem (***Approximate Nash with small support***). Assume payoffs lie in $[0, 1]$ and let $N = \max\{n, m, 2\}$. For any $\epsilon > 0$, there is a k -uniform ϵ -approximate Nash equilibrium for $k = \left\lceil \frac{12 \ln N}{\epsilon^2} \right\rceil$.

Approximating a Nash eq.

Definition (*k-uniform*). Let $k \geq 1$ be an integer. A strategy x is k -uniform if every coordinate x_i is a multiple of $1/k$.

Observation: A k -uniform strategy has support size at most k .

Theorem (*Approximate Nash with small support*). Assume payoffs lie in $[0, 1]$ and let $N = \max\{n, m, 2\}$. For any $\epsilon > 0$, there is a k -uniform ϵ -approximate Nash equilibrium for $k = \left\lceil \frac{12 \ln N}{\epsilon^2} \right\rceil$.

Remarks:

Lipton, Markakis and Mehta use the probabilistic method.

Time from enumerating pairs: $N^{O\left(\frac{\log N}{\epsilon^2}\right)}$

Rubinstein (ETH for PPAD): for some fixed $\epsilon > 0$,

$N^{\log^{1-o(1)} N}$ time is necessary. Nearly matches the dependence on N .